Informe Técnico

Rediseño BBDD

Integrantes

Ignacia Castillo
Milan Kurte
Francisco Polo

Resumen ejecutivo **3**
1. Esquema de base de datos rediseñado **5**
1.1 Método de análisis 5
1.2 Esquema original frente al rediseñado 5
1.3 Tabla de decisiones por campo 6
1.4 Alternativas para reducir el nivel de detalle 7
1.5 Cómo contribuye el nuevo diseño a la privacidad 8
2. Seudonimización y anonimización **9**
2.1 Diferencia entre ambos conceptos 9
2.2 Dónde se aplica cada técnica 9
2.3 Implementación de la seudonimización 10
2.4 Implementación de la anonimización 10
2.5 Registros, reportes, pruebas y ambientes de desarrollo 11
2.6 Restricciones de acceso a la información que revierte la seudonimización 11
3. Modelo de control de acceso basado en roles (RBAC) **13**
3.1 Roles y responsabilidades 13
3.2 Matriz de permisos 13
3.3 Acciones y datos que cada rol no debe tener 14
3.4 Mínimo privilegio y control de operaciones sensibles 14
4. Implementación en el código **16**
4.1 Validaciones: no almacenar datos innecesarios ni inválidos 16
4.2 Protección de campos sensibles, contraseñas y claves 16
4.3 Autenticación y autorización basadas en roles 17
4.4 Restricciones en la base de datos 18
4.5 Manejo seguro de errores y registros 18
4.6 Retención y supresión 18
4.7 Cambios que debe adaptar el resto del proyecto 19
5. Evidencias de pruebas **20**
5.1 Resultado por prueba 20
5.2 Casos de acceso permitido y denegado 20
5.3 Salida de ejecución 21
5.4 Comprobaciones de protección de datos 22
6. Conclusión técnica **22**
6.1 Mejoras alcanzadas 22
6.2 Limitaciones de la solución 23
6.3 Riesgos que aún deben mitigarse 23
Referencias **24**

Resumen ejecutivo

NortiShop es un comercio electrónico que gestiona clientes registrados, compras de invitados, tarjetas de pago, un boletín de marketing y análisis del comportamiento de compra. En la actividad anterior (Item II) se protegieron los datos con hashing, cifrado AES-256-GCM, tokenización e índices HMAC. Esa etapa respondió a la pregunta ¿Cómo protegemos lo que guardamos?. Este informe responde a las preguntas que vienen antes y después: ¿Necesitamos guardarlo? y ¿Quién puede leerlo una vez protegido?.

El trabajo se organizó en cuatro frentes, cada uno con cambios reales en el código del repositorio:

- **Minimización:** eliminamos el RUT, hicimos opcionales el teléfono y la dirección, y agregamos plazos de retención. El perfilamiento de compras pasó a ser opt-in (requiere consentimiento).
- **Seudonimización y anonimización:** el comportamiento de compra se guarda bajo un seudónimo (HMAC) sin tabla de correspondencia; los reportes de ventas solo entregan datos agregados por mes y región, con supresión de grupos pequeños ($k \geq 5$).
- **RBAC:** cuatro roles de personal (administrador, soporte, logística, marketing) más el cliente final, con permisos mínimos, denegación por defecto y una bitácora de auditoría que no puede alterarse.
- **Endurecimiento de la implementación:** validación de entradas, bloqueo por intentos fallidos, mensajes de error uniformes y filtro que impide que correos, RUT o números de tarjeta lleguen a los logs.

Los controles se verifican con 26 pruebas automatizadas (sección 5) que cubren tanto los accesos permitidos como los rechazados. Un resultado importante del diseño es que el rol administrador no puede leer datos personales: administra cuentas y permisos, pero no descifra contactos ni direcciones. Esto separa las funciones de quien administra el sistema de quien opera sobre los datos.

Resumen de controles

Control	Qué logra	Dónde está
Eliminación del RUT	Reduce el daño de una filtración (identificador usable para suplantación)	db.py, registrar_cliente
Campos opcionales y validaciones	Solo se almacena lo que el usuario entrega y con formato correcto	nortishop.py (validar_*)
Retención y supresión	Dirección y contacto de invitado se borran tras entregar el pedido; el cliente puede eliminar su cuenta	aplicar_retencion, eliminar_cuenta
Seudonimización	El análisis de hábitos no revela de quién son	seudonimo_cliente, comportamiento_compra
Anonimización	Reportes sin datos personales y con k-anonimato	reporte_ventas
RBAC + auditoría	Mínimo privilegio y trazabilidad de cada acceso	rbac.py, tabla auditoria

Autenticación robusta	Bloqueo por fuerza bruta y respuestas que no revelan si el usuario existe	_verificar_con_bloqueo
Logs seguros	Los registros no contienen datos personales	FiltroDatosPersonales

1. Esquema de base de datos rediseñado

1.1 Método de análisis

Cada campo del esquema original se evaluó con tres preguntas: (1) ¿qué función del sistema depende de él?, (2) ¿qué daño produciría si se filtrara? y (3) ¿existe una forma de cumplir la misma función con menos datos o con datos menos precisos? Según la respuesta, el campo se mantuvo, se eliminó, se redujo/generalizó, se hizo opcional o se reemplaza por un dato no identificable. Los campos nuevos solo se agregaron cuando son necesarios para un control de seguridad o para cumplir un principio de privacidad.

Los campos de pago merecen una mención aparte: el número de tarjeta, el vencimiento y el CVV nunca se almacenan. Se envían a la pasarela y NortiShop solo conserva el token, los últimos cuatro dígitos y la marca. Esa decisión ya estaba tomada y se mantiene.

1.2 Esquema original frente al rediseñado

La tabla resume cada tabla antes y después. El signo − indica un campo eliminado, + un campo nuevo, y ~ un campo que cambia de obligatorio a opcional o de identificable a seudonimizado. El código completo está en db.py.

Tabla	Esquema original	Esquema rediseñado
clientes	id, nombre, correo_cifrado, correo_indice, password_hash, rut_cifrado, telefono_cifrado, direccion_cifrado, nortipuntos	id, nombre, correo_cifrado, correo_indice, password_hash, telefono_cifrado ~, direccion_cifrado ~, nortipuntos − rut_cifrado + consentimiento_analitica, intentos_fallidos, bloqueado_hasta
tarjetas_guardadas	id, cliente_id, token_pasarela, ultimos4, marca	Sin cambios. (PAN, vencimiento y CVV nunca se guardan.)
pedidos	id, codigo, cliente_id, es_invitado, invitado_correo_cifrado, invitado_telefono_cifrado, direccion_cifrado (NOT NULL), detalle_cifrado, monto, token_pasarela, ultimos4, autorizacion, fecha	Mismos campos, con invitado_telefono_cifrado ~ opcional y direccion_cifrado ~ que puede quedar NULL tras la retención + region_envio, estado (pagado | despachado | entregado)
administradores	id, usuario, password_hash	id, usuario, password_hash + rol (administrador | soporte | logistica | marketing; por defecto soporte) + intentos_fallidos, bloqueado_hasta
boletin_suscriptores	id, correo_cifrado, correo_indice	Sin cambios.
comportamiento_compra	id, cliente_id (FK a clientes), datos_cifrado	id, seudonimo ~ (sin FK), datos_cifrado − cliente_id
auditoria	(no existía)	+ Tabla nueva: id, fecha, actor_tipo, actor_id, rol, permiso, recurso, resultado. Inmutable por triggers.

Tabla 1. Comparación del esquema original y el rediseñado.

1.3 Tabla de decisiones por campo

Cada fila justifica la decisión por su necesidad funcional y por el riesgo de conservar el dato. Los colores indican la decisión: verde = mantener, rojo = eliminar, amarillo = opcional, azul = reducir/seudonimizar, morado = campo nuevo.

Tabla y campo	Decisión	Justificación (necesidad funcional y riesgo)
clientes: id	Mantener	Clave técnica sin significado personal. Necesaria para relacionar tablas.
clientes: nombre	Mantener (con límite de largo)	Se usa para personalizar la cuenta y rotular el despacho. Es dato personal de bajo riesgo por sí solo. Se limita a 80 caracteres para evitar que se use como campo libre.
clientes: correo_cifrado	Mantener (cifrado AES-256-GCM)	Es el medio de contacto y el identificador de la cuenta. Sin él no hay notificaciones ni recuperación de acceso. Cifrado para que una copia de la base no lo exponga.
clientes: correo_indice	Mantener (HMAC-SHA256)	Permite buscar y autenticar por correo sin descifrar la tabla. Riesgo: al ser determinista, dos tablas con el mismo correo muestran el mismo índice; por eso se calcula con una llave que nunca está en la base.
clientes: password_hash	Mantener con protección adecuada (bcrypt)	Imprescindible para autenticar. Se almacena solo como hash con sal única y costo 12; nunca en texto plano ni cifrado reversible.
clientes: rut_cifrado	Eliminar	No interviene en vender, cobrar ni despachar. Las boletas electrónicas pueden emitirse con el correo, y si en el futuro se requiere RUT para facturar, debe pedirse en ese momento y no guardarse en la cuenta. Riesgo: es un identificador único y permanente en Chile; en una brecha facilita suplantación de identidad y cruce con otras bases.
clientes: telefono_cifrado	Opcional	El correo basta para operar. El teléfono solo agrega comodidad (aviso de despacho). Exigirlo es recolección excesiva; si el cliente no lo entrega, no se guarda (NULL).
clientes: direccion_cifrado	Opcional	Solo se necesita al despachar. El cliente puede no guardarla e indicarla en cada compra. Riesgo: ubicación física del domicilio.
clientes: nortipuntos	Mantener	Saldo del programa de fidelidad; no identifica a la persona. Su riesgo es de integridad (alteración del saldo), que se cubre con RBAC y no con cifrado.
clientes: consentimiento_analitica	Nuevo	Registra que el cliente aceptó el análisis de su comportamiento (por defecto 0). Sin este valor en 1 no se guarda ningún dato de comportamiento.
clientes: intentos_fallidos, bloqueado_hasta	Nuevo	Controles de seguridad contra fuerza bruta. No contienen datos personales.
tarjetas_guardadas: token_pasarela	Mantener	Permite cobrar sin conocer la tarjeta. Es un seudónimo emitido por la pasarela: sin ella, el token no sirve.
tarjetas_guardadas: ultimos4, marca	Mantener	Permiten que el cliente reconozca su tarjeta (“**** 4242”). Los últimos cuatro dígitos no permiten reconstruir ni usar la tarjeta.

tarjetas_guardadas: cliente_id	Mantener	Vincula la tarjeta con su dueño. Se elimina junto con la cuenta.
pedidos: codigo, monto, fecha, autorizacion	Mantener	Necesarios para contabilidad, conciliación con la pasarela y reclamos. No identifican a la persona.
pedidos: cliente_id	Mantener (pasa a NULL al eliminar la cuenta)	Permite mostrar el historial al cliente. Al ejercer el derecho de supresión, el pedido queda sin vínculo con la persona.
pedidos: es_invitado	Mantener	Distingue compras sin cuenta, que se tratan con otro plazo de retención.
pedidos: invitado_correo_cifrado	Mantener con retención limitada	Se necesita para confirmar la compra y atender reclamos. Se borra (NULL) cuando el pedido lleva más de 365 días entregado.
pedidos: invitado_telefono_cifrado	Opcional	No es indispensable para entregar el pedido; con el correo basta. Se borra con la retención.
pedidos: direccion_cifrado	Mantener con retención limitada	Indispensable para despachar. Una vez entregado y vencido el plazo, no cumple ninguna función y se borra (la columna deja de ser NOT NULL).
pedidos: detalle_cifrado	Mantener (cifrado)	El historial y la garantía requieren saber qué se compró. Los productos comprados pueden revelar hábitos o condiciones personales, por eso se cifran.
pedidos: token_pasarela, ultimos4	Mantener	Necesarios para reembolsos y para mostrar la tarjeta usada en la compra.
pedidos: region_envio	Nuevo (dato generalizado)	Reemplaza la necesidad de usar la dirección en estadísticas. Solo guarda la región (16 valores posibles), no calle ni comuna, y permite reportes por zona sin datos personales.
pedidos: estado	Nuevo	Permite que logística acceda a direcciones solo mientras el envío está activo, y no después.
administradores: usuario, password_hash	Mantener	Autenticación del personal. Contraseña con bcrypt.
administradores: rol	Nuevo	Base del RBAC. El valor por defecto es el de menor privilegio (soporte) y una restricción CHECK impide roles inventados.
administradores: intentos_fallidos, bloqueado_hasta	Nuevo	Bloqueo por fuerza bruta en cuentas privilegiadas.
boletin_suscriptores: correo_cifrado, correo_indice	Mantener	Necesarios para enviar la campaña y para no duplicar suscriptores. Solo el rol marketing puede descifrar la lista.
comportamiento_compra: cliente_id	Reducir: reemplazar por seudónimo	Un historial atado al id es un perfil directo. Con el seudónimo (HMAC) el análisis sigue siendo posible, pero la tabla ya no apunta a la persona.
comportamiento_compra: datos_cifrado	Mantener (solo con consentimiento)	Contiene categorías de interés, no el historial detallado. Solo se escribe si el cliente aceptó.
auditoria (tabla completa)	Nuevo	Evidencia quién accedió a qué. No guarda datos personales: solo ids internos, permiso y resultado.

Tabla 2. Decisión y justificación para cada campo del esquema.

1.4 Alternativas para reducir el nivel de detalle

Dato	Forma original	Alternativa adoptada o propuesta
RUT	Se guardaba cifrado en la cuenta	Eliminado. Si una factura lo exige, se pide en el momento y no se almacena.
Ubicación de la compra	Dirección completa en cada pedido, para siempre	Se agrega region_envio para estadísticas y se borra la dirección tras la retención.
Fecha de compra en reportes	Fecha y hora exactas	Se generaliza a mes (substr(fecha,1,7)) en los reportes.
Perfil de comportamiento	Historial por cliente identificado	Categorías de interés, bajo seudónimo, solo con consentimiento.
Teléfono	Solicitado siempre	Opcional en clientes y en invitados.
Tarjeta	Número completo	Token + últimos 4 dígitos (ya implementado con la pasarela).
Boletín	Solo correo, sin ciclo de vida	Propuesto: doble confirmación y enlace de baja que borre el registro (no implementado; ver sección 6).

Tabla 3. Formas de reducir la cantidad o el detalle de los datos.

1.5 Cómo contribuye el nuevo diseño a la privacidad

- **Menos datos que proteger.** Un dato que no existe no puede filtrarse. El RUT y el perfilamiento sin consentimiento desaparecen del alcance de un posible ataque.
- **Datos con fecha de vencimiento.** Direcciones y contactos de invitados no se acumulan indefinidamente: la función aplicar_retencion los elimina del pedido una vez entregado y transcurrido el plazo.
- **Derecho de supresión funcional.** eliminar_cuenta borra al cliente, sus tarjetas y su comportamiento, y deja los pedidos sin vínculo ni dirección.
- **Consentimiento explícito** para el análisis de hábitos, en lugar de recolección por defecto.
- **Alineación normativa.** La Ley N° 21.719, que reforma la Ley 19.628 y fue publicada el 13 de diciembre de 2024, entra en vigencia el 1 de diciembre de 2026 (existe un proyecto de ley, boletín 18.623-07, que propone postergarla al 1 de diciembre de 2027; al 25 de octubre de 2026 sigue en primer trámite en el Senado, donde según reportes la Comisión de Constitución lo aprobó en general el 19 de octubre, y como no es ley la fecha vigente continúa siendo el 1 de diciembre de 2026). Exige medidas técnicas proporcionales al riesgo, como seudonimización, cifrado y controles de acceso, que son las que aquí se implementan.

2. Seudonimización y anonimización

2.1 Diferencia entre ambos conceptos

	Seudonimización	Anonimización
Qué hace	Sustituye identificadores directos (correo, id) por códigos. Existe una forma de volver a vincularlos si se dispone de información adicional (una llave o una tabla).	Transforma los datos de modo que no se pueda identificar razonablemente a la persona, incluso cruzándolos con otras fuentes.
¿Sigue siendo dato personal?	Sí. Reduce el riesgo, pero el dato sigue siendo potencialmente identificable.	No, si la anonimización es efectiva. Si puede revertirse o cruzarse, no lo es.
¿Reversible?	Sí, con la información adicional.	No debería serlo.
Técnicas usadas aquí	HMAC del id y del correo; token de la pasarela.	Generalización (mes, región), agregación, supresión de grupos pequeños ($k \geq 5$) y borrado.
Cuándo conviene	Cuando el sistema todavía necesita vincular registros con una persona, o analizar por individuo sin saber quién es.	Cuando basta con estadísticas, reportes o ambientes de prueba.

Tabla 4. Seudonimización frente a anonimización.

Esta distinción es relevante también en términos legales: la Ley 21.719 define el dato anonimizado como aquel en que se eliminó el nexo con la persona y señala que deja de ser dato personal, mientras que el dato seudonimizado sigue sujeto a la ley.

2.2 Dónde se aplica cada técnica

Tabla, proceso o módulo	Técnica	Mecanismo	Qué protege	Por qué es adecuada
clientes y boletín: correo_indice	Seudonimización	HMAC-SHA256 del correo con una llave separada	Permite buscar y autenticar sin tener el correo en claro en la tabla de búsqueda	Se necesita igualdad exacta (¿existe este correo?), que el HMAC permite; sin la llave no puede recalcularse.
tarjetas_guardadas y pedidos: token_pasarela	Seudonimización	Token emitido por la pasarela de pago	Número de tarjeta (PAN)	El comercio no necesita el PAN para cobrar. La correspondencia token ↔ tarjeta la guarda un tercero certificado.
comportamiento_compra: seudonimo	Seudonimización	HMAC del id del cliente, sin tabla de correspondencia	Vínculo entre el perfil de hábitos y la persona	Marketing puede analizar patrones por individuo sin saber quién es; solo el sistema, con la llave, puede recalcular el seudónimo.
Módulo de reportes: reporte_ventas	Anonimización	Generalización (mes, región), agregación y supresión de grupos con menos de 5 pedidos	Cliente, correo, dirección, token, fecha exacta	Un reporte de ventas solo requiere totales; no hay razón para exponer personas.

Retención: aplicar_retencion	Anonimización (borrado)	Se pone en NULL dirección y contacto de invitado en pedidos entregados hace más de 365 días	Datos que ya no cumplen función	El pedido queda como registro contable sin vínculo con un lugar ni una persona.
Cuenta eliminada: eliminar_cuenta	Anonimización	Se borra el cliente y se desvincula el pedido (cliente_id = NULL, dirección = NULL)	Identidad del cliente	Cumple el derecho de supresión conservando solo lo necesario para contabilidad.
Logs y bitácora	Anonimización / seudonimización	Filtro que enmascara correo, RUT y tarjeta; la auditoría usa ids internos	Datos personales en archivos de log	Los logs suelen tener acceso más amplio que la base de datos y duran más tiempo.

Tabla 5. Aplicación de seudonimización y anonimización por componente.

2.3 Implementación de la seudonimización

El seudónimo del comportamiento de compra se calcula con el HMAC del id del cliente, reutilizando la llave de índice ya existente con un prefijo de dominio (seudonimo:), de modo que nunca coincide con un índice de correo. La tabla comportamiento_compra no tiene clave foránea a clientes ni columna cliente_id.

```
def seudonimo_cliente(cliente_id):
    # HMAC del id con la llave de índice; no hay tabla de correspondencia.
    return indice_busqueda(f"seudonimo:{cliente_id}")

def guardar_comportamiento(cliente_id, datos):
    # Solo con consentimiento y bajo seudónimo (la tabla no tiene cliente_id).
    ...
    if c is None or not c["consentimiento_analitica"]:
    return False
    seud = seudonimo_cliente(cliente_id)
    con.execute(
    "INSERT OR REPLACE INTO comportamiento_compra "
    "(seudonimo, datos_cifrado) VALUES (?,?)",
    (seud, cifrar_json(datos, f"comportamiento.datos:{seud}")))
```

El contenido sigue cifrado y su contexto de cifrado (AAD) usa el seudónimo, por lo que un registro no puede copiarse hacia otro seudónimo. Marketing consume los datos con leer_analitica, que devuelve únicamente seudonimo y datos.

Ejemplo. Para la cliente de prueba Ana (id 1) el sistema guarda un registro bajo un seudónimo de 64 caracteres hexadecimales (p. ej. 303c16ed…) con {"categorias": ["tecnologia"]}. Marketing puede contar cuántos seudónimos prefieren tecnología, pero no puede saber que ese seudónimo es Ana. Para descubrirlo, alguien debería poseer la llave y calcular el HMAC de cada id de cliente hasta encontrar la coincidencia.

2.4 Implementación de la anonimización

El reporte de ventas aplica tres transformaciones: **generalización** (la fecha se reduce al mes y la dirección a la región), **agregación** (solo cantidades y totales) y **supresión de grupos pequeños** con un umbral k = 5. La consulta nunca selecciona cliente_id, correos, direcciones ni tokens.

```
@requiere_permiso("reportes.leer")
```

```
def reporte_ventas(sesion):
        con = conectar()
        filas = con.execute(
        """SELECT substr(fecha,1,7) AS mes, COALESCE(region_envio,'Sin región') AS region,
                COUNT(*) AS pedidos, SUM(monto) AS total
        FROM pedidos GROUP BY mes, region""").fetchall()
        return [dict(f) for f in filas if f["pedidos"] >= UMBRAL_K]
```

La supresión de grupos pequeños evita que una región con uno o dos pedidos permita deducir quién compró. En las pruebas (T20), una región con 5 pedidos aparece en el reporte y otra con 2 pedidos no aparece.

Límite honesto. El k-anonimato no garantiza anonimato absoluto. Un atacante con información externa (por ejemplo, saber que solo una persona de cierta región compró un mes determinado y que el grupo se cerró justo en 5) podría inferir datos. Por eso el umbral es configurable y el reporte no incluye otros atributos cruzados.

2.5 Registros, reportes, pruebas y ambientes de desarrollo

Ámbito	Medida	Estado
Registros de actividad (logs)	El módulo rbac.py define FiltroDatosPersonales, que reemplaza correos (j***@mail.cl), RUT ([RUT]) y secuencias de 13 a 19 dígitos ([TARJETA]) antes de escribir. Además, el código solo registra ids internos (“cliente registrado id=7”), nunca valores.	Implementado y probado (T25, T26)
Mensajes de error	Las excepciones de validación nombran el campo, no el valor recibido. Los accesos denegados devuelven siempre “Acceso denegado”, tanto si el recurso no existe como si no está autorizado.	Implementado y probado (T03, T17)
Reportes	reporte_ventas entrega datos anonimizados. resumen_pedidos entrega código, monto, estado y fecha, sin datos personales, a soporte, logística y administración.	Implementado y probado (T13, T20)
Bitácora de auditoría	Guarda actor, rol, permiso, id del recurso y resultado. Nunca guarda correos ni direcciones.	Implementado y probado (T23)
Pruebas	Las pruebas usan una base temporal, datos ficticios y no tocan nortishop.db.	Implementado
Desarrollo y pruebas del equipo	Política: nunca copiar una base de producción a desarrollo. Se usa demo.py (datos ficticios) y llaves distintas por ambiente. verificar_secretos.py comprueba que no haya datos personales en claro ni llaves expuestas.	Política + script existente

Tabla 6. Prevención de exposición de datos personales por ámbito.

2.6 Restricciones de acceso a la información que revierte la seudonimización

La seudonimización solo protege si lo que permite revertirla está más protegido que los datos seudonimizados. En NortiShop, esa información es la llave HMAC y, para los tokens de pago, la bóveda de la pasarela:

Información que permite revertir	Quién puede acceder	Controles
Llave del índice HMAC (correo_indice y seudonimo)	Solo el proceso de la aplicación	Se carga desde .env (permisos 600, fuera del repositorio). Ningún rol de la matriz tiene el permiso claves.leer ni analitica.reidentificar. Si falta, el programa no inicia.
Llaves AES-256 (versionadas)	Solo el proceso de la aplicación	Mismo mecanismo. Rotación mediante NORTISHOP_LLAVE_CIFRADO_V2.
Secreto de la pasarela	Solo el proceso de la aplicación	Se usa para verificar la firma de las respuestas de pago.
Tabla de correspondencia de seudónimos	No existe	Se eligió HMAC para no tener una tabla que proteger. El costo es que revertir un seudónimo exige recalcular el HMAC de cada cliente (viable con la llave, inviable sin ella).
Vault de tokens de tarjeta	Pasarela de pago (tercero)	NortiShop nunca posee la correspondencia token ↔ PAN.
Datos descifrados puntuales	Soporte (contacto) y logística (dirección)	Un registro a la vez, con permiso explícito y entrada en la bitácora. No existe operación de descifrado masivo.

Tabla 7. Control de acceso a la información que permite revertir la seudonimización.

3. Modelo de control de acceso basado en roles (RBAC)

3.1 Roles y responsabilidades

El sistema distingue a las personas **usuarias** (cliente final) del **personal** de NortiShop. Cada persona de personal tiene exactamente un rol, almacenado en administradores.rol; el cliente se identifica por su sesión y solo accede a lo propio.

Rol	Responsabilidad	Necesita
Cliente (usuario final)	Comprar, ver su historial, gestionar su tarjeta y su cuenta, eliminar su cuenta	Sus propios datos, pedidos y tarjeta
Administrador	Crear cuentas de personal y asignar roles, revisar la auditoría, ejecutar la política de retención, ver reportes	Gestión de cuentas y visión agregada. No necesita datos personales.
Soporte	Atender consultas y reclamos de clientes	Contacto (correo y teléfono) del cliente o del invitado, y el resumen del pedido
Logística	Preparar y despachar pedidos, actualizar su estado	Dirección de despacho de pedidos activos y cambio de estado
Marketing	Enviar campañas del boletín y analizar tendencias	Lista del boletín (para enviar), analítica seudonimizada y reportes anonimizados

Tabla 8. Roles del sistema.

3.2 Matriz de permisos

Acción o recurso	Admin.	Soporte	Logística	Marketing	Cliente
Gestionar roles y cuentas de personal	Permitido	Denegado	Denegado	Denegado	Denegado
Consultar datos y pedidos propios	No aplica	No aplica	No aplica	No aplica	Permitido
Consultar contacto (correo, teléfono) de un cliente registrado	Denegado	Permitido (auditado)	Denegado	Denegado	Solo el propio
Consultar contacto de un invitado	Denegado	Permitido (auditado)	Denegado	Denegado	Denegado
Consultar dirección de despacho	Denegado	Denegado	Permitido solo en envíos activos	Denegado	Solo la propia
Actualizar estado de un pedido	Denegado	Denegado	Permitido	Denegado	Denegado
Resumen de pedidos (sin datos personales)	Permitido	Permitido	Permitido	Denegado	Denegado
Reportes de ventas anonimizados	Permitido	Denegado	Denegado	Permitido	Denegado

Analítica de comportamiento (seudonimizada)	Denegado	Denegado	Denegado	Permitido	Denegado
Descifrar la lista del boletín para enviar campaña	Denegado	Denegado	Denegado	Permitido	Denegado
Reidentificar un seudónimo	Denegado	Denegado	Denegado	Denegado	Denegado
Leer la bitácora de auditoría	Permitido	Denegado	Denegado	Denegado	Denegado
Ejecutar la política de retención	Permitido	Denegado	Denegado	Denegado	Denegado
Registrar tarjeta y comprar con tarjeta guardada	Denegado	Denegado	Denegado	Denegado	Permitido
Eliminar cuenta	Denegado	Denegado	Denegado	Denegado	Solo la propia
Acceder a claves criptográficas	Denegado (solo el sistema)	Denegado	Denegado	Denegado	Denegado

Tabla 9. Matriz de permisos. En la implementación se materializa en el diccionario `PERMISOS` de `rbac.py`.

Cada celda "Permitido" corresponde a un permiso nombrado (por ejemplo despacho.direccion.leer). Lo que no figura en el diccionario está denegado, de modo que agregar un rol o una función nueva no concede acceso por omisión.

3.3 Acciones y datos que cada rol no debe tener

Rol	No debe poder	Razón
Administrador	Descifrar contactos, direcciones o la lista del boletín; ver analítica por seudónimo	Quien administra cuentas no necesita ver datos de clientes. Si su cuenta se compromete, el atacante no obtiene datos personales con ese solo acceso.
Soporte	Ver la dirección de despacho; crear o modificar cuentas; enviar campañas; leer la auditoría	Para atender consultas basta el contacto. La dirección expone el domicilio.
Logística	Ver correo o teléfono; ver direcciones de pedidos ya entregados; ver reportes	Solo necesita la dirección mientras entrega. Después no hay motivo para consultarla.
Marketing	Ver contactos de clientes o invitados, direcciones o pedidos; identificar a quién pertenece un seudónimo	Su función es comunicar y analizar tendencias, no atender a personas concretas.
Cliente	Ver datos o pedidos de otro cliente; acceder a cualquier función de personal	Evita la lectura de datos ajenos cambiando un identificador (IDOR).
Todos	Leer llaves criptográficas; descifrado masivo; modificar o borrar la auditoría	Las llaves son del proceso de la aplicación; la bitácora es de solo agregar.

Tabla 10. Restricciones explícitas por rol.

3.4 Mínimo privilegio y control de operaciones sensibles

- **Denegado por defecto.** tiene_permiso devuelve falso para cualquier sesión inválida, rol desconocido o permiso no listado.

- **Un solo punto de control.** El decorador `@requiere_permiso("...")` envuelve cada función que toca información sensible. No hay forma de llamar a la función protegida sin pasar por la verificación.
- **Control por recurso, no solo por rol.** Un cliente solo puede leer lo suyo (`sesion.id == cliente_id`); logística solo accede a direcciones de pedidos en estado `pagado` o `despachado`.
- **Descifrado puntual y auditado.** Contacto y dirección se descifran de a un registro, justo cuando se necesitan, y cada acceso (permitido o denegado) queda en `auditoria`.
- **Operaciones administrativas separadas.** Crear cuentas exige el permiso `roles.gestionar`. El primer administrador solo puede crearse en el arranque del sistema y la función se rechaza si ya existe uno (`crear_primer_admin`).
- **Rol por defecto de menor privilegio.** Si no se indica rol, la cuenta nueva es `soporte`, y una restricción CHECK de la base impide valores fuera de los cuatro roles.
- **Bitácora inmutable.** Triggers en la base de datos abortan cualquier UPDATE o DELETE sobre `auditoria`, incluso si el código de la aplicación fuera comprometido.
- **Autenticación del personal con bloqueo temporal** tras 5 intentos fallidos, para frenar ataques de fuerza bruta contra cuentas privilegiadas.

4. Implementación en el código

Los cambios se concentran en cuatro archivos del repositorio. config.py, cripto.py, pasarela_simulada.py, generar_llaves.py y verificar_secretos.py no se modifican: la implementación reutiliza sus funciones (cifrar, descifrar, indice_busqueda, hashear_password, etc.).

Archivo	Cambio	Parte de la actividad
db.py	Esquema rediseñado: sin RUT, campos opcionales, rol, estado, region_envio, seudónimo, auditoría, restricciones CHECK y triggers de inmutabilidad	1, 2, 3
rbac.py **(nuevo)**	Roles y permisos, clase Sesion, decorador @requiere_permiso, bitácora, filtro de logs y enmascarado	2, 3, 4
nortishop.py	Validaciones, seudonimización, anonimización, retención, supresión, login con bloqueo y todas las funciones sensibles protegidas por permisos	1, 2, 3, 4
pruebas_seguridad.py **(nuevo)**	26 pruebas automatizadas de los controles	4

Tabla 11. Archivos modificados.

4.1 Validaciones: no almacenar datos innecesarios ni inválidos

La función registrar_cliente ya no recibe el RUT (si alguien intenta pasarlo, Python lanza TypeError). El teléfono y la dirección son opcionales: si no llegan, se guardan como NULL y no se cifra nada. Todas las entradas se validan antes de tocar la base de datos, y los errores nombran el campo pero nunca incluyen el valor.

```
def registrar_cliente(nombre, correo, password, telefono=None, direccion=None,
                      consentimiento_analitica=False):
    nombre = validar_opcional(nombre, "nombre", 80)
    correo = validar_correo(correo)
    validar_password(password)                    # 10 a 128 caracteres, no trivial
    telefono = validar_opcional(telefono, "teléfono", 16, _RE_TELEFONO)
    direccion = validar_opcional(direccion, "dirección", 200)
    ...
    _cifrar_opcional(telefono, _ctx_cliente("telefono", idx)),   # None si no se entregó
    _cifrar_opcional(direccion, _ctx_cliente("direccion", idx)),
```

La región de envío se valida contra la lista oficial de 16 regiones (validar_region). Así, el campo generalizado no puede usarse como un campo libre donde alguien escriba una dirección completa.

4.2 Protección de campos sensibles, contraseñas y claves

- **Contraseñas:** bcrypt (costo 12, sal única) para clientes y personal. Se exige una longitud mínima y se rechazan claves triviales.

- **Campos sensibles:** AES-256-GCM con contexto tabla.columna:identificador, de modo que un dato cifrado copiado a otra fila no se puede descifrar (prueba T05).
- **Claves:** se leen desde .env (permisos 600, fuera del repositorio) y el programa no inicia si falta alguna. El formato v1:<base64> permite rotarlas sin perder datos antiguos.
- **Pagos:** la tarjeta nunca se guarda; la respuesta de la pasarela se acepta solo si su firma HMAC es válida y coincide con el pedido y el monto.

Este nivel ya existía desde el Item II. Lo nuevo en esta actividad es que ningún rol puede usar las llaves directamente y que el descifrado de cada dato sensible exige un permiso.

4.3 Autenticación y autorización basadas en roles

Las funciones login_cliente y login_admin devuelven un objeto Sesion (tipo, id y rol) o None. Toda función que accede a información sensible recibe esa sesión como primer parámetro y la verifica con el decorador:

```
PERMISOS = {
        "soporte": {"pedidos.resumen.leer", "contacto.cliente.leer",
           "contacto.invitado.leer"},
        "logistica": {"pedidos.resumen.leer", "despacho.direccion.leer",
            "pedidos.estado.actualizar"},
        "marketing": {"boletin.enviar", "analitica.leer_seudonimizada",
                "reportes.leer"},
        "administrador": {"roles.gestionar", "pedidos.resumen.leer", "reportes.leer",
                "auditoria.leer", "retencion.ejecutar"},
        ...
}

def requiere_permiso(permiso, recurso_en_arg1=False):
        def decorador(fn):
        @wraps(fn)
        def envoltura(sesion, *args, **kwargs):
        recurso = args[0] if (recurso_en_arg1 and args) else None
        if not tiene_permiso(sesion, permiso):
        _auditar(sesion, permiso, recurso, "denegado")
        raise AccesoDenegado("Acceso denegado")
        _auditar(sesion, permiso, recurso, "permitido")
        return fn(sesion, *args, **kwargs)
        return envoltura
        return decorador

@requiere_permiso("despacho.direccion.leer", recurso_en_arg1=True)
def direccion_despacho(sesion, pedido_id):
        ...
        if f is None or f["estado"] == "entregado" or f["direccion_cifrado"] is None:
        raise AccesoDenegado("Acceso denegado")   # igual que "no autorizado"
        return descifrar(f["direccion_cifrado"],
                _ctx_pedido("direccion", f["codigo"]))
```

El control se hace en el servidor, dentro de las funciones de servicio, es decir, el código que ejecuta cualquier interfaz que se construya sobre NortiShop. Esa capa debería traducir AccesoDenegado a un código HTTP 403. Las reglas que dependen del recurso (el

cliente solo ve lo suyo, logística solo ve envíos activos) están en la propia función, no en la interfaz.

El login ofrece dos protecciones adicionales: bloqueo temporal de 15 minutos tras 5 intentos fallidos y respuestas uniformes, de modo que un correo inexistente y una clave incorrecta dan el mismo resultado y tardan lo mismo (se verifica un hash de relleno cuando el usuario no existe).

4.4 Restricciones en la base de datos

Además del control en el código, la base impone reglas que no dependen de que la aplicación se comporte bien:

```
rol TEXT NOT NULL DEFAULT 'soporte'
        CHECK (rol IN ('administrador','soporte','logistica','marketing')),

estado TEXT NOT NULL DEFAULT 'pagado'
        CHECK (estado IN ('pagado','despachado','entregado')),

CREATE TRIGGER IF NOT EXISTS auditoria_sin_update BEFORE UPDATE ON auditoria
BEGIN SELECT RAISE(ABORT, 'auditoria es inmutable'); END;
CREATE TRIGGER IF NOT EXISTS auditoria_sin_delete BEFORE DELETE ON auditoria
BEGIN SELECT RAISE(ABORT, 'auditoria es inmutable'); END;
```

Las consultas usan siempre parámetros (?), lo que evita inyección SQL. Los nombres de tabla y columna que se interpolan en _verificar_con_bloqueo provienen del código, nunca de datos del usuario.

Limitación. SQLite no tiene usuarios ni permisos por tabla, así que no es posible dar a cada rol de la aplicación una cuenta de base de datos distinta. En un despliegue con PostgreSQL o MySQL, la recomendación es crear una cuenta por función con GRANT mínimos (por ejemplo, la cuenta de reportes solo con SELECT sobre una vista anonimizada).

4.5 Manejo seguro de errores y registros

```
class FiltroDatosPersonales(logging.Filter):
        # Última línea de defensa: un dato personal escrito por error no llega al disco.
        def filter(self, record):
        record.msg = limpiar_texto(record.getMessage())
        record.args = ()
        return True
```

- Los logs de la aplicación solo mencionan ids internos y eventos (“cuenta bloqueada temporalmente”), y el filtro reemplaza correos, RUT y números de tarjeta si algún mensaje futuro los incluyera.
- Los errores de validación no repiten el valor recibido; los accesos denegados devuelven un mensaje fijo.
- Los bloqueos y las denegaciones quedan en auditoria, que sí permite investigar un incidente.

4.6 Retención y supresión

```
@requiere_permiso("retencion.ejecutar")
def aplicar_retencion(sesion, dias=365):
        # Pedidos entregados hace más de N días: se borran dirección y contacto.
        # Queda lo contable (monto, fecha, región, código).
        ...
        UPDATE pedidos SET direccion_cifrado=NULL, invitado_correo_cifrado=NULL,
                invitado_telefono_cifrado=NULL
        WHERE estado='entregado' AND fecha < ?
```

aplicar_retencion debe ejecutarse periódicamente (por ejemplo, con una tarea programada diaria que use una cuenta de administrador). eliminar_cuenta implementa la supresión a petición del cliente.

4.7 Cambios que debe adaptar el resto del proyecto

Como las funciones sensibles ahora exigen una sesión, demo.py debe actualizarse: login_cliente y login_admin devuelven Sesion, crear_admin y las consultas sensibles reciben la sesión como primer argumento, y registrar_cliente ya no acepta RUT. Las pruebas de pruebas_seguridad.py muestran el uso correcto de cada función. La base de datos existente debe recrearse (demo.py ya lo hace) porque el esquema cambió.

```
python generar_llaves.py                # solo la primera vez
python -m unittest pruebas_seguridad -v        # 26 pruebas, usan una BD temporal
```

5. Evidencias de pruebas

Las pruebas están en pruebas_seguridad.py y se ejecutan con python -m unittest pruebas_seguridad -v. Crean una base de datos temporal con clientes, personal de cada rol y pedidos ficticios, y verifican tanto los accesos permitidos como los rechazados. Resultado de la última ejecución: **26 pruebas, 26 aprobadas, 0 fallos**.

5.1 Resultado por prueba

ID	Qué verifica	Resultado
T01	Registro solo con datos mínimos: teléfono y dirección quedan NULL	Aprobada
T02	El RUT no existe en el esquema ni se acepta como parámetro	Aprobada
T03	Correo, contraseña y teléfono inválidos se rechazan sin exponer el valor	Aprobada
T04	Volcado crudo de la BD: no aparecen correos, teléfonos ni direcciones en claro	Aprobada
T05	Copiar un dato cifrado a otra fila hace fallar el descifrado (contexto AAD)	Aprobada
T06	La región de envío debe pertenecer a la lista oficial	Aprobada
T07	Login válido entrega sesión; clave errónea y correo inexistente dan el mismo resultado	Aprobada
T08	Tras 5 intentos fallidos la cuenta se bloquea, incluso con la clave correcta	Aprobada
T09	crear_primer_admin se rechaza cuando ya existe un administrador	Aprobada
T10	Sesiones inválidas (None, dict, rol inventado) son rechazadas	Aprobada
T11	Cliente lee sus datos; los de otro cliente y su historial ajeno se deniegan	Aprobada
T12	Logística lee la dirección de un envío activo, no el contacto, y no de uno entregado	Aprobada
T13	Soporte lee contacto (sin dirección) y resumen sin datos personales; la dirección de despacho se deniega	Aprobada
T14	Marketing envía boletín y ve reportes; contactos y direcciones se deniegan	Aprobada
T15	Administrador crea cuentas; no puede descifrar contacto ni direcciones ni enviar boletín	Aprobada
T16	Soporte, logística, marketing y clientes no pueden crear cuentas ni leer la auditoría	Aprobada
T17	Pedido inexistente y pedido no autorizado producen el mismo error (sin filtrar existencia)	Aprobada
T18	Sin consentimiento no se guarda comportamiento; con consentimiento sí	Aprobada
T19	La tabla de comportamiento no contiene cliente_id y marketing recibe solo seudónimos	Aprobada
T20	El reporte agrega por mes y región y suprime grupos con menos de 5 pedidos	Aprobada
T21	Pedidos entregados hace más de un año pierden dirección y contacto de invitado	Aprobada
T22	Derecho de supresión: se borra al cliente; sus pedidos quedan sin cliente ni dirección	Aprobada
T23	Cada acceso, permitido o denegado, queda en la bitácora sin datos personales	Aprobada
T24	Ni UPDATE ni DELETE son posibles sobre la bitácora (triggers en la base de datos)	Aprobada
T25	El filtro de logs reemplaza correos, RUT y números de tarjeta antes de escribir	Aprobada
T26	Registrar y autenticar clientes no deja correos en los logs	Aprobada

Tabla 12. Pruebas automatizadas de los controles (T01–T26).

5.2 Casos de acceso permitido y denegado

Los casos más representativos de la matriz RBAC, con el resultado obtenido al ejecutarlos:

Quién	Operación	Resultado esperado	Prueba
Logística	Leer dirección de un pedido pagado	Permitido	T12
Logística	Leer dirección de un pedido entregado	Denegado	T12
Logística	Leer contacto del invitado	Denegado	T12
Soporte	Leer contacto de un invitado o cliente	Permitido, sin la dirección del cliente	T13
Soporte	Leer dirección de despacho	Denegado	T13
Marketing	Descifrar boletín y ver reportes	Permitido	T14
Marketing	Leer contacto o dirección	Denegado	T14
Administrador	Crear cuentas de personal	Permitido	T15
Administrador	Leer contacto, dirección, boletín o analítica	Denegado	T15
Soporte, logística, marketing, cliente	Crear cuentas o leer la auditoría	Denegado	T16
Cliente	Leer sus datos / datos o historial de otro cliente	Permitido / Denegado	T11
Cualquiera	Sesión nula, forjada o con rol inexistente	Denegado	T10

Tabla 13. Accesos permitidos y denegados verificados.

5.3 Salida de ejecución

Extracto de la ejecución de python -m unittest pruebas_seguridad -v:

```
Ran 26 tests in 1.74s

OK
```

Comprobación de protección de datos y accesos con una base de ejemplo (filas crudas, accesos permitidos y denegados y bitácora resultante):

```
== clientes (fila cruda)
{'id': 1, 'nombre': 'Ana Prueba', 'correo_cifrado': 'v1:zITGlXeB+5nSbhZ0oPvd5CfQ3yvxID2…', 'telefono_cifrado':
'v1:DccFMOIjMIXKAt9/E8AXNhg6bJ+kaQ/…', 'direccion_cifrado': None, 'consentimiento_analitica': 1}
== comportamiento_compra (fila cruda)
303c16ed245fad426455680f… v1:jZVoKup7TwM0YBCo8kYmA…
== logistica -> direccion_despacho: Av. Falsa 123, Santiago
== soporte -> direccion_despacho: AccesoDenegado('Acceso denegado')
== logistica -> datos_contacto_invitado: AccesoDenegado('Acceso denegado')
== administrador -> datos_contacto_invitado: AccesoDenegado('Acceso denegado')
== marketing -> datos_contacto_invitado: AccesoDenegado('Acceso denegado')
== marketing -> leer_analitica: 303c16ed245fad42… {'categorias': ['tecnologia']}
== auditoria:
  logistica despacho.direccion.leer 1 permitido
  soporte despacho.direccion.leer 1 denegado
  logistica contacto.invitado.leer 1 denegado
  administrador contacto.invitado.leer 1 denegado
  marketing contacto.invitado.leer 1 denegado
  marketing analitica.leer_seudonimizada None permitido
  marketing analitica.leer_seudonimizada None permitido
  administrador auditoria.leer None permitido
```

Se observa que el correo y el teléfono aparecen solo como texto cifrado (v1:…), que la dirección no guardada es None, que el comportamiento de compra se almacena bajo un seudónimo, que los accesos no autorizados producen siempre AccesoDenegado('Acceso denegado') y que cada intento, permitido o denegado, queda registrado en la bitácora.

5.4 Comprobaciones de protección de datos

Comprobación	Cómo se verificó	Prueba
No hay datos personales en claro en la base	Se vuelca toda la base (iterdump) y se busca el correo, teléfono y dirección de prueba	T04
El cifrado está amarrado a la fila	Se copia un teléfono cifrado a otro cliente y el descifrado falla	T05
Entradas inválidas se rechazan sin filtrar el valor	Correo, clave y teléfono inválidos; el mensaje de error no contiene el valor	T03
El comportamiento exige consentimiento y va seudonimizado	Sin consentimiento no se guarda; la tabla no tiene cliente_id	T18, T19
Reporte anonimizado con k ≥ 5	Región con 5 pedidos aparece; región con 2 no aparece; solo hay campos agregados	T20
Retención y supresión	Pedido antiguo pierde dirección y contacto; el reciente no; eliminar_cuenta desvincula pedidos	T21, T22
Bitácora inmutable	UPDATE y DELETE sobre auditoria abortan	T24
Logs sin datos personales	El filtro enmascara correo, RUT y tarjeta; registrar y fallar logins no deja correos en el log	T25, T26
Bloqueo por fuerza bruta	Tras 5 fallos, incluso la clave correcta es rechazada	T08

Tabla 14. Comprobaciones de protección de datos.

6. Conclusión técnica

6.1 Mejoras alcanzadas

- **Menos exposición.** Un atacante que obtenga una copia de la base ya no encuentra RUT; teléfono, dirección y datos de invitados son opcionales o temporales; y el perfilamiento solo existe con consentimiento y bajo seudónimo.
- **Acceso mínimo y verificable.** Cada rol puede hacer solo lo que su función requiere, el rol administrador queda separado de los datos personales y 26 pruebas demuestran que los accesos no autorizados se rechazan.
- **Trazabilidad.** Cada acceso a información sensible, permitido o denegado, queda en una bitácora que ni la aplicación puede alterar.
- **Privacidad en los márgenes del sistema.** Logs, errores, reportes y bitácora no contienen datos personales.
- **Mejor posición frente a la Ley 21.719**, que exige medidas técnicas como seudonimización, cifrado y control de acceso, y que el responsable pueda demostrar que estaban operando (la bitácora y las pruebas sirven como evidencia).

6.2 Limitaciones de la solución

Limitación	Detalle
RBAC en la aplicación, no en el motor	SQLite no distingue usuarios de base de datos. Quien tenga acceso directo al archivo .db y a las llaves puede leer todo, sin pasar por el RBAC.
Llaves y datos en el mismo servidor	Una ejecución remota de código en el servidor expondría a la vez la base y las llaves del .env.
Objeto Sesion sin firma	Dentro del mismo proceso, el código podría construir una Sesion con un rol arbitrario. Fuera del proceso, la sesión debería viajar como token firmado y con expiración.
No hay capa web	El proyecto es una biblioteca con scripts de demostración. La traducción de AccesoDenegado a HTTP 403, el manejo de sesiones y el límite de peticiones por IP no están implementados.
Índices HMAC deterministas	Dos registros con el mismo correo generan el mismo índice; quien tenga la llave de índice puede confirmar si un correo está registrado. La llave de índice se reutiliza para los seudónimos (separados por prefijo).
k-anonimato limitado	Con k = 5 y solo dos atributos el riesgo de reidentificación es bajo pero no nulo. No se aplicó privacidad diferencial.
Filtro de logs basado en patrones	Reconoce correos, RUT y números largos, pero no nombres ni direcciones escritas en texto libre; la defensa principal sigue siendo no registrarlos.
Bloqueo por intentos	Puede usarse para bloquear a propósito una cuenta ajena. Para clientes conviene complementar con CAPTCHA o límites por IP.
Retención manual	aplicar_retencion no se ejecuta sola; requiere programar la tarea.

Tabla 15. Limitaciones.

6.3 Riesgos que aún deben mitigarse

Riesgo pendiente	Mitigación recomendada
Compromiso total del servidor (RCE)	Guardar las llaves en un servicio de gestión de llaves (KMS/HSM), mover la base a un servidor dedicado en red privada, y usar cuentas de base de datos distintas por función con GRANT mínimos.
Robo de credenciales del personal	Autenticación multifactor obligatoria, sesiones con expiración corta y revisión periódica de la bitácora.
Abuso interno legítimo (soporte con acceso a contactos)	Alertas ante consultas anómalas (por ejemplo, muchas consultas por hora) y revisión periódica de la auditoría. Hoy se registra pero no se alerta.
Pérdida de una llave	Procedimiento de respaldo y rotación de llaves; la versión v1: ya permite migrar a v2 sin perder datos antiguos.
Datos en respaldos y copias	Aplicar la misma retención y el mismo cifrado a los respaldos; no restaurarlos en ambientes de desarrollo.
Boletín sin ciclo de vida	Doble confirmación de suscripción y enlace de baja que elimine el registro.
Emergencias que requieran acceso administrativo a datos	Definir un procedimiento “de emergencia” (doble aprobación y registro reforzado), dado que el rol administrador no puede leer datos personales por diseño.

Tabla 16. Riesgos residuales y mitigaciones.

En síntesis, el rediseño reduce lo que se guarda, limita quién puede leerlo y deja evidencia de cada acceso. Los riesgos que quedan se concentran en la infraestructura (llaves y servidor) y en la capa de sesiones, que exceden el alcance de esta etapa pero tienen una ruta de mitigación clara.

Referencias

- OWASP. Application Security Verification Standard (ASVS), capítulos de autenticación, control de acceso y registro de eventos.
- OWASP. Password Storage Cheat Sheet y Authorization Cheat Sheet.
- ANSI/INCITS 359-2004. Role Based Access Control (modelo estándar de RBAC).
- PCI Security Standards Council. PCI DSS: prohibición de almacenar el código de seguridad (CVV) tras la autorización y reglas de protección del PAN.
- Sweeney, L. (2002). k-anonymity: a model for protecting privacy. International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems.
- Chile. Ley N° 21.719, que regula la protección y el tratamiento de los datos personales y crea la Agencia de Protección de Datos Personales (Diario Oficial, 13 de diciembre de 2024).
- Reglamento (UE) 2016/679 (RGPD): art. 4(5) seudonimización, art. 25 protección de datos desde el diseño y por defecto, art. 32 seguridad del tratamiento y considerando 26.